

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 08

বক্তৃতা – 38

বাবার ওভারফ্লো আক্রমণ ব্যাহত করা

(Preventing Buffer Overflow Attacks)

পূর্ববর্তী ভিডিওতে, আমরা দেখেছি যে কিভাবে attacker রা মূলত একটি সিস্টেমের unauthorized user তার সিস্টেমকে একটি buffer overflow bug হিসাবে পরিচিত বলে exploit করে তার কোডটি চালাতে পারে। তাই আমরা দেখেছি, অন্তত আমরা এই ধরনের একটি buffer overflow bug বাগ তৈরি করেছি এবং দেখিয়েছি যে attacker কীভাবে তার কোডটি ব্যবহার করতে পারে এবং এই কোডটি ব্যবহার করে সিস্টেমের shell তৈরি করে। সুতরাং, এই বিশেষ উদাহরণ user space একটি অ্যাপ্লিকেশন ভিত্তিক উদাহরণ ছিল, কিন্তু অপারেটিং সিস্টেমে খুব অনুরূপ ধরনের exploit করা যেতে পারে। তাই, এই ভিডিওতে আমরা যা দেখব তা এই কৌশলগুলি কিভাবে এই buffer overflow দুর্বলতা অতিক্রম করে, এবং কিভাবে এই বছরের পর বছর attack গুলি প্রগতি করেছে এবং বছরের পর বছর ধরে উন্নত হয়েছে, এই সুরক্ষাগুলি কাটিয়ে উঠতে আরো শক্তিশালী হয়েছে।

(স্লাইড সময় : 01:32 দেখুন)

Stack এর মধ্যে যে buffer overflow attack টি ঘটে তা প্রতিরোধ করার প্রথম এবং সবচেয়ে সুস্পষ্ট উপায় হল stack page গুলি non-executable ভাবে তৈরি করা। আমরা যা দেখেছি তা হল যে attacker CPU চালানোর জন্য জোর করে একটি কোড ব্যবহার করে যা stack এর মধ্যে উপস্থিত রয়েছে। উদাহরণস্বরূপ, এই buffer টি

stack এর উপর সংজ্ঞায়িত করা হয়, এই কোডটিও exploit কোড এবং string copy টি এমনভাবে প্রয়োগ করা হয়েছিল যে string copy টি তার execution সম্পন্ন হওয়ার পরে, stack এর shell code যেটি exploit কোড সেটি হবে। সুতরাং, এই attack টি প্রতিরোধ করার একটি সুস্পষ্ট উপায় হল stack এর page গুলিকে non-executable করা, তাই এবং এটি আজকালকার সিস্টেমগুলিতে করা হয়। তাই, যদি আপনি আসলে আপনার Intel সিস্টেমে এই নির্দিষ্ট প্রোগ্রামটি চালান, এবং কোডটি চালানোর কাজে ব্যবহার করার পরিবর্তে, আপনি একটি segmentation fault পেতে পারেন। এই কারণ হতে পারে কারণ প্রোগ্রাম stack এর উপর কিছু instructions চালানোর চেষ্টা করছে।

(স্লাইড সময় : 02:58 দেখুন)

Intel মেশিনগুলিতে, stack কে non-executable হিসাবে চিহ্নিত করার জন্য page table এ একটি NX বিট উপস্থিত রয়েছে। বেশিরভাগ প্রোগ্রামে এটি কাজ করে তবে এটি একটি অতিরিক্ত সুবিধা, তবে সমস্যা হল কিছু প্রোগ্রাম যদিও malicious না হওয়া সত্ত্বেও stack থেকে execute করার দরকার হতে পারে। তাই এই প্রোগ্রাম সঠিকভাবে কাজ করার জন্য stack থেকে চালানো প্রয়োজন। অতএব, NX বিট সেটিং সব প্রোগ্রামের জন্য সবসময় খুব উপকারী হয় না।

(স্লাইড সময় : 03:37 দেখুন)

তাই পরবর্তী জিনিস, আমরা নিজেদেরকে জিজ্ঞাসা করতে যাচ্ছি যে আমরা যদি stack কে non-executable হিসাবে তৈরি করি তবে কি এটি সম্পূর্ণভাবে buffer overflow attack প্রতিরোধ করবে, এবং প্রকৃতপক্ষে তা হয় না। বছরের পর বছর ধরে buffer overflow attack গুলি return to libc attack এ পরবর্তিত হয়েছে, যা এমন সিস্টেমগুলিতেও ব্যবহার করা যেতে পারে যেখানে non-executable stack আছে।

(স্লাইড সময় : 04:13 দেখুন)

সুতরাং আসুন আমরা খুব সংক্ষিপ্তভাবে দেখি যে কিভাবে return to libc attack কাজ করে। সুতরাং, মূলত আমরা কি করেছি, যখন আমরা stack এর buffer টি প্রবাহিত করার চেষ্টা করি তখন আমরা stack এর উপরে উপস্থিত কোডটি ব্যবহার করেছিলাম, এবং আমরা buffer address দিয়ে valid return address টি প্রতিস্থাপিত করেছি। এখন এটি আমাদের জন্য আধুনিক দিনের সিস্টেমে কাজ করে না, কারণ stack non-executable এ সেট করা হয়েছিল। তাই আসুন আমরা দেখি কিভাবে return to libc কাজ করে, non-executable stack থাকা সত্ত্বেও, অর্থাৎ NX bit থাকা সত্ত্বেও।

(স্লাইড সময় : 04:56 দেখুন)

সুতরাং, return to libc টি কি তা হল stack এর মধ্যে একটি location যাওয়ার পরিবর্তে, এটি return address কে অন্য কোন ফাংশন আছে এমন অন্য কোন location এ নিয়ে যাবে। আসুন আমরা এই ফাংশনটি F1 বলি। তাই, একটি buffer overflow এর মাধ্যমে stack এ যা পূরণ হয় তা হল F1 এর একটি pointer সুতরাং যখন ফাংশনটি execute করা হয়, এখানে থেকে নেওয়া return address টি F1 এ একটি pointer এবং সেইজন্য, CPU এই F1 ফাংশন execution এ বাধ্য হয়। এখন, পরবর্তী প্রশ্ন হল - এই ফাংশন F1 টি কি? সুতরাং, একটি জিনিস নিশ্চিত যে এই ফাংশন attacker এর নিজস্ব exploit code কোড হতে পারে না। সুতরাং, এটিকে কিছু বৈধ ফাংশন হতে হবে যা code segment এ ইতিমধ্যে বিদ্যমান এবং যা execute করার অনুমতি আছে। সুতরাং, এই ফাংশন F1 কি হবে? 1 নম্বর point এবং 2 নম্বর point হল একটি attacker কিভাবে প্রোগ্রামে উপস্থিত একটি সাধারণ ফাংশন ব্যবহার করে কিছু malicious code যেমন exploit code execute করে?

(স্লাইড সময় : 06:23 দেখুন)

বিভিন্ন উপায়ে ফাংশন F1 প্রয়োগ করা যায়, কিন্তু আমরা আজ যা দেখব তা হচ্ছে এই বিশেষ ফাংশন system ব্যবহার করে ফাংশন F1 এর প্রয়োগ। এখন, system এমন একটি ফাংশন যা libc নামে লাইব্রেরিতে উপস্থিত থাকে এবং এটি কীভাবে একটি character pointer কে একটি string এ নেয় যা executable গ্রহণ করবে। সুতরাং string যা system এর কাছে passed হয় একটি executable নাম নেয় এবং যখন এটি execute করা হয় তখন এই বিশেষ প্রোগ্রামটি চালানো হবে। সুতরাং, এই বিশেষ উদাহরণে, system কি করে এটি bash shell execute করে একটি bash shell তৈরি করে। এখন libc একটি লাইব্রেরী, যা বেশিরভাগ প্রোগ্রাম দ্বারা ব্যবহৃত হয়। সুতরাং, এমনকি একটি সাধারণ "Hello World" প্রোগ্রাম যেটি আপনি লিখেছেন সম্ভবত libc লাইব্রেরি ব্যবহার করবে। সুতরাং, এর অর্থ কি আপনার প্রসেসের address location এ, ফাংশন system টি আছে। সুতরাং, attacker কে এখন কি করতে হবে তা কেবল আপনার প্রসেসের address space সনাক্ত করতে হবে যেখানে system ফাংশন থাকবে। পরবর্তীতে তিনি একটি প্রোগ্রাম execute করার জন্য একরকম একটি argument pass প্রয়োজন। সুতরাং, আমরা যদি উদাহরণটিই দেখতে থাকি অব্যাহত আছে, attacker একটি exploit তৈরি করে shell execute করে যা পূর্ববর্তী ভিডিওতে দেখেছি। সুতরাং, এই ক্ষেত্রে এছাড়াও, যদি attacker একই shell execute করতে চান, তারপর memory space এ system ফাংশনের address খুঁজে পাওয়া ছাড়াও, attacker কোনভাবে এই নির্দিষ্ট string slash bin slash sh কে parameter হিসাবে pass করতে হবে। সুতরাং, এর মানে হল যে attacker কে একটি address খুঁজতে হবে যা string slash bin slash sh এর দিকে point করে।

(স্লাইড সময় : 08:32 দেখুন)

সুতরাং, সাধারণত কি করা হয় এর মত একটি stack frame রয়েছে যেখানে return address থাকে বা valid return address টি F1 pointer দ্বারা প্রতিস্থাপিত হয় যা system ফাংশনটির একটি pointer যা বিদ্যমান libc তে এবং যদি আপনার প্রোগ্রামটি dynamically সংযুক্ত লাইব্রেরির libc ব্যবহার করে, তাহলে এই ধরনের address valid হবে। দ্বিতীয়ত, attacker কে কী করতে হবে তা হল system ফাংশনে একটি argument pass করতে হবে, যেখানে মূলত একটি executable name রয়েছে। সুতরাং, মূলত কোথাও address space এ, attacker এর এই নির্দিষ্ট string টি পাওয়া প্রয়োজন যা slash bin slash bash বা slash bin slash sh, এবং এই নির্দিষ্ট string pointer stack টি পূরণ করা। সুতরাং, মূলত, এটি দুটি জিনিস প্রয়োজন; একটি হল stack এর return address যা system call এর address দিয়ে modify করতে হবে যা libc- এ উপস্থিত রয়েছে, এবং libc এ passed parameter টি stack এ উপস্থিত হতে হবে। সুতরাং, এই ক্ষেত্রে এটি একটি shell pointer যা এই নির্দিষ্ট string slash bin slash bash নির্দেশ করে। সুতরাং, এটি করা হয় একবার, যখন ফাংশন return করে তখন valid address এ ফেরার পরিবর্তে, এটি CPU কে এই নির্দিষ্ট system execute করতে বাধ্য করবে। এখন এই প্রক্রিয়ার সময়, এই বিশেষ shell slash bin slash bash - এ pointer টি system call এ বিবেচনা করা হবে এবং এর ফলে একটি shell তৈরি করা হবে। সুতরাং, shell থেকে, attacker তারপর অন্যান্য বিভিন্ন জিনিস spawn এবং তার নিজের প্রোগ্রাম চালাতে পারে। সুতরাং, এইভাবে, return to libc attack একটি non-executable stack থাকা সত্ত্বেও কাজ করে। সুতরাং, মনে রাখবেন আমরা এই instructions execute করি না; ফাংশন system দ্বারা code segment এর মধ্যে বাস্তব execution ঘটলেই আমরা stack এ read এবং write করতে পারি।

(স্লাইড সময় : 10:51 দেখুন)

এখন, return to libc এর সীমাবদ্ধতা হল যে কিছু arbitrary code চালানো অত্যন্ত কঠিন। সুতরাং, আমরা একটি উদাহরণ দেখেছি কিভাবে attacker একটি shell execute করতে পারে, কিন্তু attacker একটি return to libc attack দ্বারা যা করতে পারে তা খুব সীমিত। এবং অতএব, সময়ের সাথে আরও শক্তিশালী attack এসেছে।

(স্লাইড সময় : 12:24 দেখুন)

এবং আরো সম্প্রতি, কিছু Return Oriented Programming Attack হিসাবে পরিচিত বলে কিছু এসেছে। এটি সর্বাধিক শক্তিশালী attack গুলির মধ্যে একটি যা buffer overflow ব্যবহার করে। সুতরাং, এটি এমন সিস্টেমের জন্য প্রযোজ্য যেখানে non-executable stack আছে। Return Oriented Programming Attack বা সংক্ষিপ্ত ROP attack Stanford University এর Hovav Shacham দ্বারা আবিষ্কার করা হয়েছিল এবং এটি একটি

non-executable stack থাকা সত্ত্বেও কোন arbitrary computation বা কোন arbitrary code প্রোগ্রামে inject করার অনুমতি দেয়। সুতরাং আসুন একটি ছোট উদাহরণের সাথে দেখি কিভাবে এই জিনিস কাজ করে।

(স্লাইড সময় : 12:13 দেখুন)

তাই আসুন আমরা বলতে পারি যে attacker টি যে কোডটি execute করতে চায় তা এই assembly lines দ্বারা নির্ধারিত হয়। সুতরাং, মূলত যদি attacker এই assembly code execute করে, তাহলে তার কাজ সম্পন্ন করা হয়, সে যা কিছু চায় তা চালাতে সক্ষম হবে। এখন ROP attack Gadget হিসাবে পরিচিত কিছু ব্যবহার করে। এখন Gadget টি মূলত এই বিশেষ কোড বা payload, যেমন এটিকে বলা হয়, কে ছোট component, যেগুলিকে Gadget বলা হয়, এ বিভক্ত করে। সুতরাং, এটি দেখানো হয়েছে যে বিভিন্ন payload গুলি কেবল gadget গুলি ব্যবহার করেই execute করা যেতে পারে এবং তাই আমরা attacker দ্বারা ব্যবহৃত বিভিন্ন ধরনের exploit code পেতে পারি। তাহলে আসুন দেখি একটি gadget কি?

(স্লাইড সময় : 13:12 দেখুন)

সুতরাং মূলত একটি gadget কিছু দরকারী instructions এবং দরকারী instructions বলতে বলছি যে এটি payload এর এমন instructions, যার সাহায্যে একটি exploit এর অংশ হিসেবে execute করা প্রয়োজন যার পরে একটি 'ret' – return থাকে। সুতরাং, Intel instructions set এ ret এর অর্থ বিশেষ ফাংশন থেকে return। সুতরাং, এটা খুব সহজ জিনিস হল - একটি gadget কি? তাই এখন, attacker কে যা করতে হবে সেটি payload এর সাথে সংশ্লিষ্ট, যা তিনি execute করতে চান, executable এর সম্পূর্ণ binary code খুঁজে এই ধরনের দরকারী gadget গুলি বের করতে হবে।

(স্লাইড সময় : 14:00 দেখুন)

উদাহরণস্বরূপ, যদি আমরা এই বিশেষ payload টি গ্রহণ করি, তবে attacker প্রোগ্রামে যে কোনও binary খুঁজে পেতে পারেন যা instruction এর মধ্যে রয়েছে, এই বিশেষ gadget টি আছে যা প্রায় একই জিনিস করে যা প্রয়োজন অর্থাৎ এই বিশেষ payload এ উপস্থিত এই instruction mov esi comma 0x8 esi এবং এই ঠিক একই জিনিস এখানে এখানে উপস্থিত। একইভাবে, attacker কি করবে সে প্রোগ্রামের অন্যান্য অংশগুলি scan করবে যাতে এই ধরনের gadget গুলি খুঁজে পাওয়া যায়। তাই payload এ দ্বিতীয় লাইন gadget 2 এ উপস্থিত; তৃতীয় লাইন এখানে, ইত্যাদি। সুতরাং, মূলত কি তিনি করবেন এই gadget গুলি ব্যবহার করে payload টি execute করতে বাধ্য করবেন। সুতরাং, তিনি stack overflow করছেন যেগুলি একটি chain of gadgets, তাই মূলত stack এ G1, G2, G3 এবং G4 থাকবে। তাই এই gadget গুলির address গুলিতে সংগঠিত হয় এমন একটি উপায় যখন valid return address খোঁজার পরিবর্তে প্রথম valid return address পূরণ করা হয়, gadget 1 এর

address পাওয়া যায় এবং ফলস্বরূপ gadget 1 এর সাথে সংশ্লিষ্ট এই instruction execute করা হয়। এবং তারপর একটি return আছে এবং stack এইভাবে বিন্যস্ত যে এরপর gadget 2 তারপর gadget 3, gadget 4, এই ভাবে execute করা হয়। সুতরাং, সামগ্রিকভাবে যদিও instruction গুলি পরপর location এ থাকে না তবুও attacker সমস্ত instruction execute করতে সক্ষম হয়। সুতরাং, এইভাবে, তিনি তার payload টি চালাতে সক্ষম হবে এবং এটা দেখানো হয়েছে যে এই ধরনের gadget এর একটি বড় সেট প্রোগ্রামগুলিতে থাকা সম্ভব।

(স্লাইড সময় : 16:22 দেখুন)

তাই, buffer overflow এর জন্য অন্যান্য সতর্কতাগুলি হল প্রোগ্রামিং ভাষা যেমন java যা স্বয়ংক্রিয়ভাবে array limit পরীক্ষা করা। সুতরাং, এটা নিশ্চিত যে কোন array limit এর বাইরে access হবে না। আরেকটি উপায় হল আরও নিরাপদ লাইব্রেরি ব্যবহার করা। উদাহরণস্বরূপ, C 11 specification annex K, এই সুরক্ষিত লাইব্রেরিগুলিকে ব্যবহার করতে নির্দেশ করে। সুতরাং, উদাহরণস্বরূপ, get s underscore s, string copy underscore s, string n underscore s, এই সবই standard function যেগুলি ব্যবহার করা হয় তার কার্যকারিতা আছে, কিন্তু এই ফাংশন নিরাপদ এবং buffer overflow প্রতিরোধ করে।

(স্লাইড সময় : 17:14 দেখুন)

Buffer overflow প্রতিরোধের আরও একটি জনপ্রিয় উপায় হল Canaries ব্যবহার করে। সুতরাং, একটি canary একটি পরিচিত pseudo random মান যা buffer overflow গুলি সনাক্ত করার জন্য stack এর উপরে স্থাপন করা হয়। কি কাজ করা হয় যে ফাংশন শুরুতে একটি canary stack এর মধ্যে ঢোকানো হয়; এখানে যেমন প্রদর্শিত stack এ কিছু canary মান push করা হয়। সুতরাং, parameter ছাড়াও, return address, frame pointer, এখন একটি canary ও আছে। এবং ফাংশন return করার ঠিক আগে, canary টি পরীক্ষা করা হয় যে এটি modify করা হয়েছে কিনা। এখন যদি একটি buffer overflow ঘটে থাকে তবে আমরা জানি যে buffer overflow stack এর address গুলিকে modify করবে এবং ফলস্বরূপ canary এর মানটি modify করা হবে, এবং সেইজন্য, আমরা যদি একটি buffer এর মধ্যে canary এর মান পরিবর্তন সনাক্ত করতে সক্ষম হই তার অর্থ buffer overflow ঘটেছে এবং সেইজন্য, আমরা সম্ভবত প্রোগ্রাম বন্ধ করতে সক্ষম হব। সুতরাং, আজকাল gcc compiler এর সাম্প্রতিক version গুলিতে, canary default রূপে প্রয়োগ করা হয়। তাই, এটা বলা হচ্ছে যে canary ব্যবহার সম্পূর্ণভাবে বর্জন করা হয়, যদি canary এর মান জানা থাকে, তবে attacker যদি বুঝতে পারেন যে canary এর মানটি ব্যবহার করা হয় তবে তিনি কেবল এই ধরনের canary টির মান পরিবর্তন বা সেট করতে পারেন যাতে canary সব সময়ে পরিবর্তিত না হয় আর এজন্য, এর ব্যবহার সীমিত।

(স্লাইড সময় : 18:56 দেখুন)

এই attack প্রতিরোধের আরও একটি উপায় হল Address Space Randomization বা ASLR, attack আটকানোর এই পদ্ধতিতে এই ব্যাপারটি ব্যবহার করে যে attacker দেয় কোডের নির্দিষ্ট location গুলি জানতে হবে। উদাহরণস্বরূপ return to libc attack এ, attacker কে জানতে হবে program space এ কোথায় ফাংশন F1 বা আমাদের উদাহরণে কোথায় ফাংশন system সিস্টেম আছে। এখন যদি আমরা ASRL ব্যবহার করি তবে address space এর layout randomize করা হয়, attacker এর পক্ষে এটি নির্ধারণ করা কঠিন হবে যে ফাংশনটি কোথায় উপস্থিত যা exploit হওয়ার প্রয়োজন। অন্য কথায়, attacker এর পক্ষে খুঁজে বের করা কঠিন হবে address space এর কোথায় ফাংশন system উপস্থিত। সুতরাং এটি attack করা আরো কঠিন করবে।
ধন্যবাদ।